Integrated Review: Lecture 1-6

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **63-67**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Connect OS mechanisms and policies across all six lectures.

- Use the comparison tables and 30 review questions.

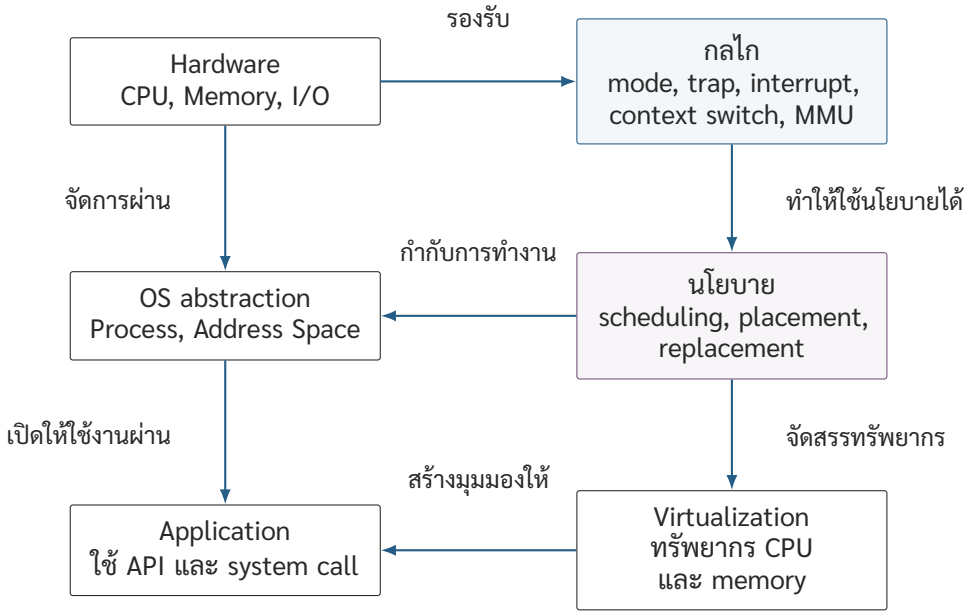
- Trace system call, context switch and page-fault flows from memory.

The original pages say FINAL REVIEW. They review the Lecture 1-6 midterm material, not later final-exam lectures.

CPE333 FINAL REVIEW

ทบทวนความเชื่อมโยงระหว่าง CPU Virtualization, Process, Scheduling, Address Translation และ Virtual Memory

R.1 แผนภาพรวมแนวคิดของ OS



ภาพนี้ช่วยให้เห็นว่าเนื้อหา 6 Lecture ต่อเนื่องกัน ไม่ได้แยกเป็นหัวข้อที่ไม่เกี่ยวข้อง:

1. Lecture 1 วางแนวคิด OS abstraction และ control boundary
2. Lecture 2 อธิบาย process และ LDE ซึ่งทำให้เกิด CPU virtualization
3. Lecture 3 ศึกษา scheduler policy สำหรับเลือกผู้ใช้ CPU
4. Lecture 4 เริ่ม memory virtualization ด้วยการแปล logical address เป็น physical address ผ่าน Base/Bounds
5. Lecture 5 ขยายไปสู่ noncontiguous translation ด้วย Segmentation, Paging และ TLB
6. Lecture 6 ใช้ disk เป็น backing store ทำให้ memory abstraction ใหญ่กว่า RAM และเชื่อม CPU, MMU, I/O, process state กับ scheduling เข้าด้วยกัน

R.2 เปรียบเทียบ Mechanism กับ Policy

หัวข้อ	Mechanism: ระบบทำงานอย่างไร	Policy: ควรเลือกอะไรหรือเมื่อใด
การควบคุม CPU	User/Kernel mode, trap, timer interrupt และ context switch	เลือก process ด้วย FIFO/ SJF/ STCF/ RR/ MLFQ/Lottery
Process state	บันทึก/ คืน ค่า PCB และ จัด ready/ block queue	เลือก process ถัดไป
Contiguous memory	แปล address ด้วย Base/Bounds และแจ้ง exception	เลือก partition และตำแหน่งวาง
Segmentation	ใช้ segment table และตรวจ bounds	เลือกพื้นที่ด้วย First/Best/Worst Fit
Paging	แปล address ด้วย Page Table, MMU และ TLB	เลือก page size และจัดการ free frame
Virtual Memory	ตรวจ present bit, page-fault trap, disk I/O และอัปเดต PTE	เลือก page เข้า/ออกและจัดสรร frame

จุดทบทวน

เมื่อถามว่า OS ทำงานอย่างไร ให้อธิบาย mechanism ที่ทำให้ทำงานนั้นได้ก่อน หากถามว่าควรเลือกวิธีใดหรือจัดสรรทรัพยากรให้ใคร ให้อธิบาย policy พร้อม metric และ trade-off ที่เกี่ยวข้อง

R.3 ลำดับการเปลี่ยน State และ Flow สำคัญ

R.3.1 System Call

User code เรียก library หรือ API ที่นำไปสู่ trap จากนั้น kernel handler ทำงานด้วยสิทธิ์ที่เหมาะสม เมื่อเสร็จแล้ว return-from-trap คืน control ให้ user code

R.3.2 Preemptive Context Switch

เมื่อ Process A กำลังรันแล้วเกิด timer interrupt OS จะบันทึก context ของ A ลง PCB จากนั้น scheduler เลือก Process B และคืน context ของ B ก่อนให้ B ทำงานต่อ

R.3.3 I/O state transition

Process ที่กำลัง Running จะเปลี่ยนเป็น **Blocked** เมื่อร้องขอ I/O และต้องรอ เมื่อ I/O เสร็จ interrupt จะแจ้ง OS ให้เปลี่ยนเป็น **Ready** จากนั้น process จะกลับเป็น Running เมื่อ scheduler เลือกให้ใช้ CPU

R.3.4 Page Fault

เมื่ออ้างอิง VA แล้วพบว่า page ไม่อยู่ใน RAM จะเกิด page-fault trap OS เริ่มอ่าน page จาก disk และเปลี่ยน process เป็น Blocked ระหว่างนั้น process อื่นใช้ CPU ได้ เมื่อ I/O เสร็จ OS อัปเดต PTE และเปลี่ยน process กลับเป็น Ready เมื่อได้รับ CPU อีกครั้งจึงคืน context และ **เริ่ม instruction เดิมใหม่**

R.4 Formula Sheet

R.4.1 Scheduling

$$T_{turnaround} = T_{completion} - T_{arrival}$$

$$T_{response} = T_{first\text{-}run} - T_{arrival}$$

R.4.2 Base / Bounds

$$VA < Bounds \Rightarrow PA = Base + VA; \qquad VA \geq Bounds \Rightarrow Fault$$

R.4.3 Segmentation

$$LA = (s, d), \quad d < Bounds_s \Rightarrow PA = Base_s + d$$

R.4.4 Paging

$$PN = \left\lfloor \frac{VA}{PageSize} \right\rfloor, \qquad offset = VA \bmod PageSize$$

$$PA = PFN \times PageSize + offset$$

Offset bits $= \log_2(PageSize)$ เมื่อ page size เป็น power of two

R.4.5 Virtual Memory

$$EAT = (1 - p_f)ma + p_f da$$

$$D = \sum_i WS_i, \qquad D > m \Rightarrow \text{thrashing condition ตาม Lecture}$$

$$TLB\ Reach = (TLB\ Size)(Page\ Size)$$

R.5 ตารางเปรียบเทียบ Scheduling

นโยบาย	metric หลัก	กฎเลือก	ข้อควรพิจารณา
FIFO	baseline เรียบง่าย	เรียงตามเวลามาถึง	งานสั้นอาจต้องรอหลังงานยาว
SJF	turnaround	เลือกงานที่สั้นที่สุด	ไม่แย่ง CPU และต้องรู้ความยาวงาน
STCF	turnaround	เลือกเวลาที่เหลือน้อยที่สุด	แย่ง CPU ได้ แต่อาจตอบสนองช้า
RR	response/fairness	หมุนตาม quantum	คำนึงถึง quantum และ switch overhead
MLFQ	turnaround และ response	ปรับ priority ตามพฤติกรรม	ใช้ boost และ accounting ป้องกัน starvation/gaming
Lottery	สัดส่วน CPU	สุ่มจาก ticket	ผลระยะสั้นแปรผันตามการสุ่ม

R.6 ตารางเปรียบเทียบการจัดการ Memory

	Contiguous: Base/Bounds	Segmentation	Paging
ตำแหน่งใน memory	หนึ่ง block ต่อ process	วางแต่ละ segment แยกกัน	วาง page ใน frame ใดก็ได้
หน่วยที่จัดสรร	ทั้ง address space	segment ขนาดไม่เท่ากัน	page/frame ขนาดเท่ากัน
การแปลงที่อยู่	Base+VA	segment Base+offset	Page Table แปลง PN เป็น PFN
การป้องกัน	ตรวจ bounds	ตรวจ bounds/ permissions ต่อ segment	ใช้ PTE/MMU
Fragmentation	ขึ้นกับ scheme	external	internal
Metadata	น้อยมาก	segment table ขนาดเล็ก	page table อาจมีขนาดใหญ่

R.7 เปรียบเทียบโครงสร้าง Page Table

โครงสร้าง	ใช้เมื่อใดหรือเพื่ออะไร	ข้อแลกเปลี่ยนหลัก
Linear	ใช้ PN เป็น index โดยตรง	ใช้ memory มากและต้องอ่าน Page Table
Two-Level	สร้างตารางระดับล่างเฉพาะส่วนที่ใช้	เพิ่มจำนวน lookup ที่ต้องทำ
Inverted	มีหนึ่ง entry ต่อ physical frame	ต้องค้น mapping ของ (PID, VPN)
Hashed	เก็บเฉพาะ mapping ที่ valid	ต้องค้น hash bucket และ chain
TLB	cache translation ที่ถูกใช้บ่อยใน hardware	มี reach จำกัด และอาจต้อง flush ตอน context switch ตามแบบจำลองใน Lecture

R.8 เปรียบเทียบ Page Replacement

นโยบาย	วิธีเลือก victim	ประเด็นที่ควรจำ
FIFO	เลือก page เก่าที่สุดใน queue	ใช้ queue และอาจเกิด Belady's anomaly
LRU	เลือก page ที่ไม่ได้ใช้นานที่สุด	ใช้ประวัติการอ้างอิงและไม่เกิด Belady's anomaly เมื่อมี stack property
LFU	เลือก page ที่มีตัวนับใช้งานต่ำสุด	หาก count เท่ากันต้องกำหนดกฎตัดสิน เช่น FIFO
OPT	เลือก page ที่จะถูกเรียกใช้อีกช้าที่สุด	ให้ faults ต่ำสุด ใช้เป็น benchmark แต่ต้องรู้อนาคต
Second Chance	FIFO ร่วมกับ reference bit	หาก ref=1 ให้ reset bit แล้วตรวจ page ถัดไป

R.9 จุดที่มักสับสน

1. **Program ต่างจาก Process**: program คือ code/data ที่จัดเก็บไว้ ส่วน process รวม state ที่ใช้ระหว่างทำงาน
2. **Ready ต่างจาก Blocked**: Ready พร้อมใช้ CPU ส่วน Blocked ต้องรอ event
3. **Procedure Call ต่างจาก System Call**: มี privilege และ control boundary ต่างกัน
4. **Trap ต่างจาก Hardware Interrupt**: trap เกิดจาก instruction หรือ exception ส่วน interrupt เป็นเหตุการณ์ภายนอกที่เกิดไม่พร้อมกับ instruction
5. **Policy ต่างจาก Mechanism**: RR/MLFQ เป็น policy ส่วน timer/context switch เป็น mechanism
6. **Context Switch ต่างจาก Process Swapping**: อย่างแรกเปลี่ยน CPU state อย่างหลังย้ายข้อมูลระหว่าง memory กับ disk
7. **Logical ต่างจาก Physical Address**
8. **Internal ต่างจาก External Fragmentation**
9. **Page ต่างจาก Frame**
10. **Page Table ต่างจาก TLB**
11. **TLB miss ต่างจาก Page Fault**
12. **Page Fault ต่างจาก protection/segmentation fault**
13. **Page Selection ต่างจาก Page Replacement**
14. **SJF ต่างจาก STCF**: SJF ไม่แย่ง CPU ส่วน STCF แย่ง CPU ได้และพิจารณา remaining time
15. **RR quantum too short is not free**: context-switch overhead can dominate

R.10 คำถามทบทวน 30 ข้อ

1. OS อยู่ระหว่าง application กับ hardware เพื่ออะไร?
2. User Mode และ Kernel Mode ต่างกันอย่างไร?
3. System Call ใช้ trap ทำอะไร?
4. ทำไม trap table ต้องถูกตั้งโดย kernel?
5. Process ต่างจาก Program อย่างไร?
6. P,D,C,S ของ Process คืออะไร?
7. PCB ต้องเก็บอะไรเพื่อ resume process?
8. Ready, Running, Blocked เปลี่ยนกันอย่างไร?
9. Limited Direct Execution มี 3 ส่วนอะไร?
10. Cooperative control พังอย่างไรเมื่อ process infinite loop?
11. Timer Interrupt ช่วย OS regain CPU อย่างไร?
12. Scheduler กับ Context Switch ต่างกันอย่างไร?
13. fork(), wait(), exec() ทำคนละอะไร?
14. Turnaround กับ Response formula ต่างกันอย่างไร?
15. FIFO drawback คืออะไร?
16. ทำไม SJF optimal เฉพาะภายใต้ assumptions บางชุด?
17. STCF ใช้ preemption อย่างไร?
18. Quantum สั้น/ยาว trade-off อย่างไร?
19. MLFQ final 5 rules คืออะไร?
20. Priority Boost แก้อะไร? Better Accounting แก้อะไร?
21. Lottery ticket 75/25 แปลว่าอะไร?
22. Logical address ต่าง physical address อย่างไร?
23. Base/Bounds translation ทำสองขั้นอะไร?
24. Static vs Dynamic relocation ต่างกันอย่างไร?
25. Internal vs External fragmentation อยู่ ตรงไหน ของ allocation?
26. Segmentation กับ Paging ต่างกันอย่างไร?
27. PN/offset แยกจาก address bit อย่างไร?
28. TLB hit/miss flow เป็นอย่างไร และทำไม miss ไม่เท่ากับ page fault?
29. Page Fault end-to-end ตั้งแต่ CPU ถึง disk แล้วกลับมา restart instruction อย่างไร?
30. Thrashing, Working Set, $D > m$ เชื่อมกันอย่างไร?

R.11 Last-Minute Trace Drills

แบบฝึก A — CPU Virtualization

ลองเขียนลำดับภายใน 60 วินาที เริ่มจาก user process ที่กำลังรัน เมื่อเกิด timer interrupt OS เข้า handler และบันทึก context ลง PCB จากนั้น scheduler เลือก process ถัดไป ก่อนคืน context เพื่อให้ทำงานต่อ

แบบฝึก B — Scheduling

กำหนด arrival time และ burst time ของ jobs แล้ววาด Gantt chart สำหรับ FIFO/SJF/STCF/RR ก่อนคำนวณ metric การวาด timeline ช่วยป้องกันการสลับ completion time กับ first-run time

แบบฝึก C — Address Translation

ก่อนทำโจทย์ให้เขียน field ของ address ก่อน: Base/Bounds = VA, Segmentation = $s|d$, Paging = $PN|offset$ และ Hybrid = $s|p|po$

แบบฝึก D — Page Replacement

สร้างตารางที่มีคอลัมน์ Reference, Frames, Fault? และ Victim ในทุก step ติดตาม queue สำหรับ FIFO, ลำดับการใช้งานสำหรับ LRU, count และกฎตัดสินสำหรับ LFU ส่วน OPT ต้องดู reference ในอนาคต

แบบฝึก E — Page Fault

คำตอบไม่ควรหยุดแค่ OS โหลด page จาก disk ให้กล่าวถึง Blocked state, การทำงานของ process อื่น, I/O completion interrupt, การอัปเดต PTE และการเริ่ม instruction เดิมใหม่ให้ครบ

> **สรุปท้ายบท**
>
> ภาพรวมที่ต้องเข้าใจก่อนสอบคือ hardware ให้กลไกที่ OS ใช้สร้าง abstraction และควบคุมทรัพยากร Process ใช้ CPU ตาม policy ของ scheduler ส่วน MMU แปลที่อยู่หน่วยความจำ และ Virtual Memory ใช้ disk เป็น backing storage เมื่อ page ที่ต้องใช้ไม่อยู่ใน RAM จะเกิด Page Fault ซึ่งเชื่อม trap, I/O, process state และ scheduling เข้าด้วยกัน
> ใช้ภาพรวมนี้ควบคู่กับการฝึกคำนวณ scheduling timeline, address translation และ page replacement เพื่อเชื่อมแต่ละ Lecture ให้เป็นระบบเดียว